



Module	Structure Machine	
Filière	MI	1 ^{ère} Année S1



Représentation de l'information. Codages & Représentations.

Le rôle de l'électronique numérique est de traiter des informations numériques représentant des grandeurs physiques (lumière, température, tension, vitesse, son, pression, nombre d'objets comptés, etc.). Mais avant de traiter une grandeur il faut la convertir en une **image numérique**, représentative de la grandeur et pouvant être manipulée par l'électronique. On utilise pour cela un **code**, qui traduit fidèlement et sans ambiguïté un nombre décimal (image de la grandeur) en une **succession de bits** appelée **mot-code** (image numérique de la grandeur). Un mot-code sera matérialisé par une succession de lignes électriques ne pouvant prendre que 2 valeurs (code binaire) en fonction de l'état de chacun des bits (exemple : bit à 0 → 0 volt sur la ligne, bit à 1 → 5 volts sur la ligne).

Ainsi codé, l'image de la grandeur pourra être analysée par des circuits numériques (circuits logiques, microcontrôleurs, calculateurs, etc.). Mais il **existe plusieurs codes binaires** (code binaire signifie que chaque élément (appelé élément binaire ou bit) ne peut prendre que 2 valeurs différents), ayant chacun leurs avantages et leurs inconvénients, et possédant des propriétés utilisées dans des applications spécifiques.

Certains sont adaptés pour le **calcul numérique**, d'autres seront utilisés pour **réaliser des capteurs de position** délivrant un code sans aléas, d'autres encore permettent **la détection et la correction d'erreurs** et seront alors utilisés pour la transmission d'informations numériques.

Lors du traitement numérique d'une information, il sera donc souvent nécessaire de passer d'un code à un autre. Les circuits logiques permettant de réaliser cette opération sont appelés **codeurs**, **décodeurs**, **encodeurs**, ou encore **transcodeurs** selon les cas.

Le champ d'application des systèmes numériques est très étendu. Lorsque l'application ne nécessite pas de calculs arithmétiques, on emploie dans certains systèmes des codes permettant d'éviter des états transitoires parasites lors de la saisie de données, ou de visualiser facilement des chiffres ou des lettres, ou bien encore de détecter des erreurs et/ou de les corriger dans un résultat susceptible d'être erroné. Nous présentons ci-après quelques codes fréquemment utilisés.

CLASSIFICATION DES CODES BINAIRES

L'ensemble des codes binaires peuvent être regroupés en deux classes : les **codes pondérés** et les **codes non pondérés**.

Un code est dit **pondéré** si la position de chaque symbole dans chaque mot correspond à un poids fixé par exemple 1, 10, 100, 1000 ... pour la numération décimale, et 1, 2, 4, 8, ... pour la numération binaire. Les codes pondérés ont, en général, des propriétés intéressantes du point de vue arithmétique : **le code binaire pur**, **le code BCD**.

Dans le cas des codes **non pondérés**, il n'y a pas de poids affecté à chaque position des symboles. On convient simplement d'un tableau de correspondance entre les objets à coder et une représentation binaire. De tels codes peuvent néanmoins parfois posséder des propriétés arithmétiques intéressantes, comme le **code excédent 3**, le **code gray**.

I. LES CODES PONDERES

I.1. LE CODE BINAIRE PUR ET SES DERIVES (octal, hexadécimal)

Ceux sont des codes qui donnent à chaque combinaison une équivalence décimale et dans laquelle chaque rang d'élément binaire a un poids précis. Le code binaire naturel et ses dérivés (octal et hexadécimal) répondent aux règles classiques de l'arithmétique des nombres positifs. Ce sont les codes utilisés en arithmétique binaire et qui ont été étudiés dans le premier chapitre.

I.2. LE CODAGE BCD (BINARY CODED DECIMAL)

Travailler sur des nombres en binaire naturel est intéressant dans les calculateurs, car ces nombres sont pondérés, mais lorsqu'on veut une image rapide de l'équivalent décimal, on est amené à effectuer un transcodage long et fastidieux. Il est plus commode dans certaines applications, comme par exemple l'affichage en décimal du contenu de compteurs, d'utiliser la représentation BCD.

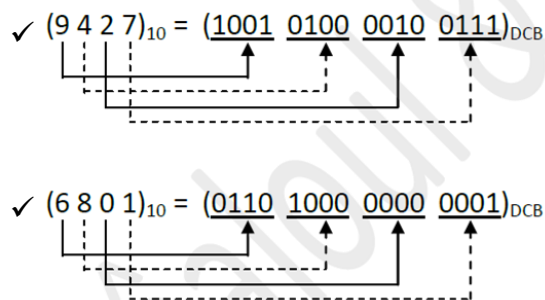
Le BCD (Binary Coded Decimal, ou Décimal Codé en Binaire en français) est le **code décimal** le plus utilisé en électronique. Il contient des mots-code qui sont la traduction en binaire naturel (sur 4 bits) de chacun des dix chiffres du système décimal.

Chiffre	Bits	Chiffre	Bits
0	0000	5	0101
1	0001	6	0110
2	0010	7	0111
3	0011	8	1000
4	0100	9	1001

Code BCD : 8421

Chaque élément binaire d'un mot-code a un poids comme en binaire naturel : 8 4 2 1. Le BCD est donc un code **pondéré**, pour retrouver un chiffre décimal à partir de son mot-code en BCD il suffit d'effectuer une conversion binaire → décimale pour chacune des tétrades composant le code BCD.

Exemples



Remarque

- ✓ Le nombre BCD est toujours en décimal !
 - Il faut donc utiliser la conversion décimale à binaire pour obtenir sa valeur en binaire.
- ✓ Le BCD n'utilise qu'une partie des codes possibles :
 - 12 bits en BCD => max 999
 - 12 bits en binaire => max $2^{12}-1 = 4095$



Opérations arithmétiques en BCD• **L'addition en BCD**

$$18_{10} + 8_{10} = 0001\ 1000 + 0000\ 1000$$

```
0001 1000
0000 1000
-----
```

```
0001 10000  On ajoute 6 car c'est > 9
```

```
  0110
0001 10110  Le cinquième bit est passé comme retenue au digit de gauche
```

```
-----
0010 0110  = 2610
```

• **Soustraction en BCD**

Pour la soustraction, on soustrait 6 au résultat si on a dû faire un emprunt pour effectuer la soustraction.

$$20_{10} - 8_{10} = 0010\ 0000 - 0000\ 1000$$

```
0010 0000
- 0000 1000
-----
```

```
0001 1000  Emprunt, on soustrait 6
   110
```

```
-----
0001 0010  = 1210
```

D'autres codes BCD existent, certains fonctionnant selon le même principe de pondération, comme le code 2421 ou le code 5421.

I.3. LE CODE AÏKEN (LE CODE 2421)

Le code **Aïken** regroupe les deux propriétés des codes BCD et XS 3 précédents : c'est un code décimal **pondéré** et **auto-complémentaire**. Les poids des éléments binaires sont 2 4 2 1. La différence entre le code **Aïken** et le code BCD est le poids du premier bit à gauche : il valait 8 en BCD alors qu'il vaut 2 en code **Aïken**.

Chiffre	Bits	Chiffre	Bits
0	0000	5	1011
1	0001	6	1100
2	0010	7	1101
3	0011	8	1110
4	0100	9	1111

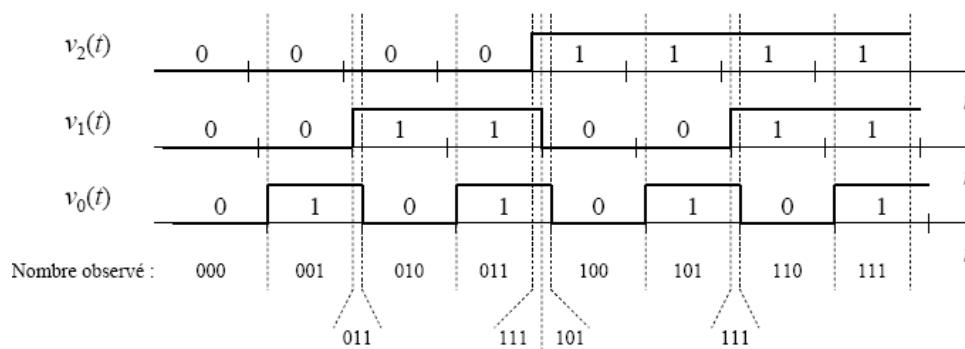
Code BCD : 2421

II. LES CODES NON PONDERES**II.1. CODE DE GRAY (BINAIRE REFLECHI)**

L'un des problèmes importants des systèmes numériques tient dans la synchronisation des différents signaux numériques. Lorsque l'on conçoit un compteur, ce problème devient important car dans la représentation binaire des nombres entiers, il arrive souvent que plusieurs bits changent en même temps. Dans ce cas, il est possible que des nombres non désirés apparaissent dans le compte. Considérons l'exemple suivant. Nous voulons concevoir un compteur trois bits, en faisant varier trois signaux, v_0 , v_1 , v_2 :



(CAS RÉEL)



Lorsque le compteur passe de 001 à 010, il doit modifier deux bits. S'il y a un problème de synchronisation entre les bits v_1 et v_0 , le nombre 011 s'affiche pendant un court laps de temps, bien que ce ne soit pas le résultat recherché.

Pour remédier à ce problème il est possible d'utiliser le code de Gray, inventé aux Bell Labs par Frank Gray et breveté en 1953. La propriété principale du code gray est que le passage d'un **mot-code** (nombre) au suivant entraînera toujours le changement d'un bit et d'un seul à la fois. Ainsi, les transitions s'effectuent sans ambiguïté, éliminant les risques d'aléas.

De plus le code est **cyclique** pour un nombre de mot-code égal à une puissance de 2. En revanche le code gray **n'est pas pondéré**, il n'est donc pas adapté pour le calcul numérique. Tout comme le binaire naturel, le binaire réfléchi peut coder n'importe quel nombre entier naturel.

Remarque : le code Gray est le code utilisé dans les tableaux de Karnaugh, afin de coder la valeur des différentes entrées de telle sorte qu'une seule d'entre elles ne change de valeur d'une ligne à l'autre, ou d'une colonne à l'autre.

On construit le code de Gray en recopiant les bits de façon symétrique (effet miroir) et en procédant de façon itérative jusqu'au bit désiré.

Décimal	Binaire	Gray
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

0	0	0	0	00
1	0	1	0	01
	1	1	0	11
	1	0	0	10
			1	10
			1	11
			1	01
			1	00

Itération 1

Itération 2

Itération 3

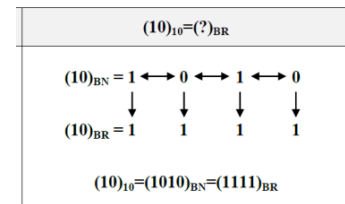
Le code de Gray est encore très utilisé, notamment dans les ordinateurs où le compteur de programme utilise le code de Gray pour minimiser sa consommation d'énergie. On le retrouve également dans certains systèmes de télécommunication pour la correction d'erreur.



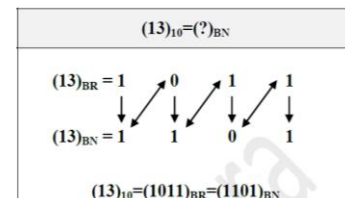
Le lien entre un mot-code n codé en binaire réfléchi et un mot-code N codé en binaire naturel et codant le même nombre est le suivant : $n = \left\lfloor \frac{N \oplus 2N}{2} \right\rfloor$

Remarque

- ✓ **Conversion du Binaire Naturel vers le Binaire Réfléchi** : il s'agit de comparer les bits B_{n+1} et le bit B_n du binaire naturel, le résultat est B_{BR} du binaire réfléchi qui vaut 0 si $B_{n+1} = B_n$ ou 1 sinon. Le premier bit à gauche reste inchangé.



- ✓ **Conversion du Binaire Réfléchi vers le Binaire Naturel** : il s'agit de comparer le bit B_{n+1} du binaire naturel et le bit B_n du binaire réfléchi le résultat est B_n du binaire naturel qui vaut 0 si $B_{n+1} = B_n$ ou 1 sinon. Le premier bit à gauche reste inchangé.



II.2. LE CODAGE EXCESS3 OU EXCEDENT 3 (CODE DE STIBITZ)

Le code à excès de trois (Excess3 noté XS 3 en abrégé) s'obtient en ajoutant 3 à chaque mot-code du code BCD. Tout comme le BCD, le code à excès de 3 est un code **décimal**, son tableau de conversion ne concerne donc que les chiffres de 0 à 9. Comme en BCD, pour coder un nombre en code à excès de trois, il faudra concaténer une succession de **tétrades**, traduisant chacune un chiffre du nombre à coder.

Le code à excès de trois a été créé pour permettre la réalisation simple des opérations de soustraction. Le complément à 1 d'un mot-code représente le complément à 9 dans l'ensemble source : les codes possédant cette propriété sont appelés des **codes auto-complémentaires**.

Exemple avec le chiffre 7 :

En XS3, le chiffre 7 se code 1010. En XS3 le **complément à 1** de 7 est donc 0101 (complément de chacun des bits). En décimal le complément à 9 de 7 est 2 ($9-7=2$). On remarque que 0101 est bien le mot-code de 2 en XS3. Et cette propriété d'auto-complémentarité peut être vérifiée pour les 10 chiffres : **Le complément à 1 d'un mot-code en XS 3 correspond au complément à 9 du chiffre en décimal.**

Décimal	BCD+3	Binaire
0	3	0011
1	4	0100
2	5	0101
3	6	0110
4	7	0111
5	8	1000
6	9	1001
7	10	1010
8	11	1011
9	12	1100

CODES ALPHANUMERIQUES (CODAGE DES CARACTERES)

Certains codes peuvent avoir une signification non numérique. Les caractères également doivent être représentés en binaire de manière unique. La convention adoptée est d'associer à chaque caractère un nombre décimal et de convertir ce nombre en binaire. Il existe plusieurs normes :

CODE EBCDIC (IBM)

Le code EBCDIC (Extended Binary Decimal Interchange Code) est un code à 8 éléments binaires utiles, soit 256 combinaisons possibles. Le code EBCDIC (Extended Binary-Coded Decimal Interchange Code), développé par IBM, permet de coder des caractères sur 8 bits. Bien que largement répandu sur les machines IBM, il n'a pas eu le succès qu'a connu le code ASCII.



CODE ASCII

Le code ASCII (American Standard Code for information interchange) est un code alphanumérique, devenu une norme internationale. Il est utilisé pour la transmission entre ordinateurs ou entre un ordinateur et des périphériques. Sous sa forme standard, il utilise 7 bits (initialement un code à 7 bits). Ce qui permet qui permet le codage de $2^7=128$ caractères. Il englobe des lettres (majuscules et minuscules), des chiffres, des signes de ponctuations et un certain nombre de signaux de commande. Toutes ces correspondances sont fixées par l'American National Standards Institutes.

Chaque code est défini par 3 bits d'ordre supérieur $b_6b_5b_4$ et 4 bits d'ordre inférieur $b_3b_2b_1b_0$. Ainsi le caractère "A" a pour code hexadécimal 41_H

Exemple :

USASCII code chart

0 1 2 3 4 5 6 7 b7 b6 b5 b4 b3 b2 b1 b0					0	1	2	3	4	5	6	7
Row \ Column	0	1	2	3	4	5	6	7				
0	0 0 0 0	0	NUL	DLE	SP	@	P	`				
1	0 0 0 1	1	SOH	DC1	!	A	Q	a				
2	0 0 1 0	2	STX	DC2	"	B	R	b				
3	0 0 1 1	3	ETX	DC3	#	C	S	c				
4	0 1 0 0	4	EOT	DC4	\$	D	T	d				
5	0 1 0 1	5	ENQ	NAK	%	E	U	e				
6	0 1 1 0	6	ACK	SYN	&	F	V	v				
7	0 1 1 1	7	BEL	ETB	'	G	W	w				
8	1 0 0 0	8	BS	CAN	(	H	X	x				
9	1 0 0 1	9	HT	EM	)	I	Y	y				
10	1 0 1 0	10	LF	SUB	*	J	Z	z				
11	1 0 1 1	11	VT	ESC	+	K	[	{				
12	1 1 0 0	12	FF	FS	,	L	\					
13	1 1 0 1	13	CR	GS	-	M	]	~				
14	1 1 1 0	14	SO	RS	.	N	^	~				
15	1 1 1 1	15	SI	US	/	O	_	DEL				

$A \Rightarrow (65)_{ASCII} \Rightarrow (01000001)_2 \Rightarrow (41)_H$
 $a \Rightarrow (97)_{ASCII} \Rightarrow (01100001)_2 \Rightarrow (61)_H$
 $[\Rightarrow (91)_{ASCII} \Rightarrow (01011011)_2 \Rightarrow (5B)_H$
 $\{ \Rightarrow (123)_{ASCII} \Rightarrow (01111011)_2 \Rightarrow (7B)_H$

Le code ASCII a été mis au point pour la langue anglaise, il ne contient donc pas de caractères accentués, ni de caractères spécifiques à une langue. Pour coder ce type de caractère il faut recourir à un autre code. Le code ASCII a donc été étendu à 8 bits (un octet) pour pouvoir coder plus de caractères $\Rightarrow$ on parle d'ailleurs de code ASCII étendu.

CODE ASCII ETENDU (8 bits)

Ce code est normalisé ISO (International Standard Organisation). C'est une extension du code ASCII. Il permet le codage de caractères sur 8 bits, soit 256 caractères possibles selon diverses normes de codage. Elles (les normes) incluent la plupart les 128 premiers codes du code ASCII, mais diffèrent dans leur usage des 128 codes restant disponibles. Parmi ces normes, la norme dite iso-latin-1 permet de représenter les caractères accentués présents dans les langues latines dont le français. Ainsi, Le code ASCII étendu n'est pas unique et dépend fortement de la plateforme utilisée.

Il nous arrive souvent d'utiliser les codes de 128 à 255 pour les accents par exemple, mais ces codes sont différents d'un pays à l'autre ! Pas pratique pour échanger des documents. Il faut donc trouver un code plus pratique. Il existe : c'est l'**UNICODE**.

UNICODE (16 bits)

Le code UNICODE permet de représenter tous les caractères spécifiques aux différentes langues. De nouveaux codes sont régulièrement attribués pour de nouveaux caractères : caractères latins (accentués ou non), grecs, cyrillics, arméniens, hébreux, thaï, hiragana, katakana... L'alphabet Chinois Kanji comporte à lui seul 6879 caractères.



Le code Unicode est un système de codage des caractères sur 16 bits mis au point en 1991. Le système Unicode permet de représenter n'importe quel caractère (caractères appartenant à plusieurs langues) par un code sur 16 bits (65536 caractères), indépendamment de tout système d'exploitation ou langage de programmation.

Il regroupe ainsi la quasi-totalité des alphabets existants (arabe, arménien, cyrillique, grec, hébreu, japonais, coréen, latin, ...) et est compatible avec le code ASCII. L'ensemble des codes Unicode est disponible sur le site <http://www.unicode.org>.

	000	001	002	003	004	005	006	007
0	NUL	SP	0	@	P		p	
1	SOH	!	1	A	Q	a	q	
2	STX	"	2	B	R	b	r	
3	ETX	#	3	C	S	c	s	
4	HT	\$	4	D	T	d	t	
5	END	%	5	E	U	e	u	
6	ACK	&	6	F	V	f	v	
7	BEL	'	7	G	W	w		
8	BS	(	8	H	X	h	x	
9	HT	)	9	I	Y	i	y	
A	LF	*	:	J	Z	j	z	
B	VT	+	;	K	[	k	{	
C	FF	.	<	L	\	l		
D	CR	-	=	M	]	m	}	
E	SO	.	>	N	^	n	~	
F	SI	/	?	O	_	o		DEL

Caractères Unicode 0000 à 007F
(0 à 127) (Caractères latins)

	008	009	00A	00B	00C	00D	00E	00F
0	CTL	CTL	À	Á	Â	Ã	Ä	Å
1	CTL	CTL	à	á	â	ã	ä	å
2	CTL	CTL	Ê	Ë	Ì	Í	Î	Ï
3	CTL	CTL	ê	ë	ì	í	î	ï
4	CTL	CTL	Ñ	Ò	Ó	Ô	Õ	Ö
5	CTL	CTL	ñ	ò	ó	ô	õ	ö
6	CTL	CTL	Ù	Ú	Û	Ü	Ý	Þ
7	CTL	CTL	ù	ú	û	ü	ý	þ
8	CTL	CTL	ß	À	Á	Â	Ã	Ä
9	CTL	CTL	à	á	â	ã	ä	å
A	CTL	CTL	Æ	Ç	È	É	Ê	Ë
B	CTL	CTL	æ	ç	è	é	ê	ë
C	CTL	CTL	Ì	Í	Î	Ï	Ð	Ñ
D	CTL	CTL	ì	í	î	ï	ð	ñ
E	CTL	CTL	Ò	Ó	Ô	Õ	Ö	×
F	CTL	CTL	ò	ó	ô	õ	ö	×

Caractères Unicode 0080 à 00FF (128 à 255)
(caractères latins, dont accentués)

	090	091	092	093	094	095	096	097
0	ए	उ	र	ी	ऊ	ऋ	ॠ	ॡ
1	ऑ	इ	ऋ	ॠ	ॡ	ॢ	ॣ	।
2	ओ	ढ	ल	ॢ	ॣ	।	॥	॥
3	ओ	ण	ळ	॥	॥	॥	॥	॥
4	ओ	त	ळ	॥	॥	॥	॥	॥
5	अ	क	थ	व	॥	॥	॥	॥
6	आ	ख	द	श	॥	॥	॥	॥
7	इ	ग	ध	॥	॥	॥	॥	॥
8	ई	घ	न	म	॥	॥	॥	॥
9	उ	ङ	त	ह	॥	॥	॥	॥
A	ऊ	च	प	॥	॥	॥	॥	॥
B	ऋ	ॠ	ॡ	ॢ	ॣ	।	॥	॥
C	ल	ज	ब	॥	॥	॥	॥	॥
D	ए	झ	भ	॥	॥	॥	॥	॥
E	ऐ	ज	म	॥	॥	॥	॥	॥
F	ए	ट	य	॥	॥	॥	॥	॥

Caractères Unicode 0900 à 097F (2304 à 2431)
(caractères devanagari)

CODE UTF-8

Même si l'UNICODE est bien conçu, il reste assez peu utilisé par rapport à l'ASCII. Pour les programmeurs, ça n'est pas toujours très facile à manipuler non plus. Ce standard se développe de plus en plus. Les langages Java, .Net (C#) et Python supportent déjà nativement l'UNICODE. La plupart des systèmes d'exploitation (Windows, Linux, MacOS X....) supportent déjà l'Unicode. Unicode, dans la théorie, c'est très bien. Mais dans la pratique, c'est une autre paire de manches. Généralement en Unicode, un caractère prend **2 octets**. Autrement dit, le moindre texte prend **deux fois plus de place qu'en ASCII**. C'est du gaspillage. De plus, si on prend un texte en français, la grande majorité des caractères utilisent seulement le code ASCII. Seuls quelques rares caractères nécessitent l'Unicode. On a donc trouvé une astuce : l'**UTF-8**.

Un texte en UTF-8 est simple : il est partout en ASCII, et dès qu'on a besoin d'un caractère appartenant à l'Unicode, on utilise un caractère spécial signalant "*attention, le caractère suivant est en Unicode*". Par exemple, pour le texte "Bienvenue chez Sébastien !", seul le "é" ne fait pas partie du code ASCII. On écrit donc en UTF-8 :

Bienvenue chez SÃ@bastien !

L'UTF-8 rassemble le meilleur de deux mondes : l'efficacité de l'ASCII et l'étendue de l'Unicode. D'ailleurs l'UTF-8 a été adopté comme norme pour l'encodage des fichiers XML. La plupart des navigateurs récents supportent également l'UTF-8 et le détectent automatiquement dans les pages HTML.



REPRESENTATION DES NOMBRES ENTIERS

Il existe deux types d'entiers :

- ✓ Les entiers non signés (positifs),
- ✓ Les entiers signés (positifs ou négatifs).

Problème : Comment indiquer à la machine qu'un nombre est négatif ou positif ?

Il existe 3 méthodes pour représenter les nombres négatifs :

- ✓ Signe/ valeur absolue.
- ✓ Complément à 1 (complément restreint).
- ✓ Complément à 2 (complément à vrai).

REPRESENTATION DES NOMBRES SIGNES (LES NOMBRES RELATIFS)

Les systèmes numériques doivent être capables de traiter des nombres positifs et négatifs. L'utilisation d'une représentation signée suppose l'utilisation d'un format (nombre de bits) fixé au préalable. Partant de ce principe, il existe différentes techniques pour représenter les nombres négatifs :

III.1.1. REPRESENTATION SIGNE / VALEUR ABSOLUE (S/VA)

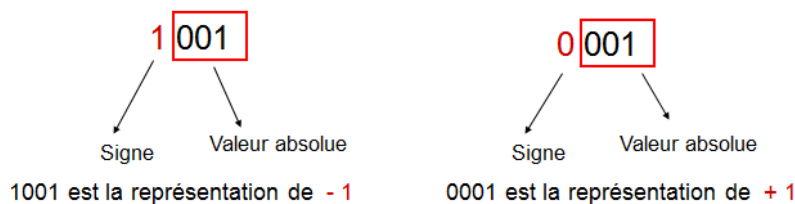
La solution la plus simple consiste à réserver un digit binaire (bit) pour **ce signe**. Les autres bits représentant une valeur absolue du nombre. La convention retenue impose de mettre le premier bit (qu'on appelle aussi bit de poids fort) à 0 pour repérer un nombre positif et à 1 pour un nombre négatif. On parle alors de donnée signée quand on utilise cette convention.

Toutefois, une telle représentation des nombres signés entraînerait un traitement spécial du signe et des circuits électroniques différents selon que l'on veuille réaliser des divisions ou des soustractions.

Si on travaille sur n bits, alors

- Le bit du poids fort est utilisé pour indiquer le signe :
 - ✓ 1 : signe négatif
 - ✓ 0 : signe positif
- Les autres bits (n-1) désignent la valeur absolue du nombre.

Exemple



Sur 3 bits on obtient :

- Les valeurs sont comprises entre -3 et +3
 - ✓ $-3 \leq N \leq +3$
 - ✓ $-(4-1) \leq N \leq +(4-1)$
 - ✓ $-(2^2-1) \leq N \leq +(2^2-1)$
 - ✓ $-(2^{(3-1)}-1) \leq N \leq +(2^{(3-1)}-1)$

Ainsi d'une manière générale, si on travaille sur n bits, l'intervalle des valeurs qu'on peut représenter en S/VA :

$$-(2^{(n-1)}-1) \leq N \leq +(2^{(n-1)}-1)$$

Avantages et inconvénients

- ✓ C'est une représentation assez simple.
- ✓ On remarque que le zéro possède deux représentations +0 et -0 ce qui conduit à des difficultés au niveau des opérations arithmétiques.

signe	VA	valeur
0	00	+ 0
0	01	+ 1
0	10	+ 2
0	11	+ 3
1	00	- 0
1	01	- 1
1	10	- 2
1	11	- 3



- ✓ Pour les opérations arithmétiques il nous faut deux circuits : l'un pour l'addition et le deuxième pour la soustraction.
- ✓ L'idéal est d'utiliser un seul circuit pour faire les deux opérations, puisque $a - b = a + (-b)$

L'inconvénient de cette représentation est résolu par une autre forme de représentation des nombres négatifs dite représentation **en complément** ou encore représentation sous **forme complémentée**.

REPRESENTATION EN COMPLEMENT

La représentation des nombres sous la forme en complément se fait selon deux modes qui ne s'appliquent en principe qu'aux nombres négatifs. Le premier mode est le **complément restreint** (ou complément à un) et le second est le **complément vrai** (ou complément à deux). Le complément restreint d'un nombre binaire s'obtient par la simple inversion des valeurs des bits. 1100 devient 0011. Le complément vrai d'un nombre s'obtient en ajoutant 1 au complément restreint.

REPRESENTATION EN COMPLEMENT A UN (COMPLEMENT RESTREINT)

Pour trouver le complément à un d'un nombre (noté C_1) appelé aussi complément logique, il suffit d'inverser tous les bits de ce nombre ($0 \rightarrow 1$ et $1 \rightarrow 0$): si le bit est un 0 mettre à sa place un 1 et si c'est un 1 mettre à sa place un 0.

Exemple

Sur 4 Bits	Sur 5 Bits
1 0 1 0	0 1 0 1 0
↓ ↓ ↓ ↓	↓ ↓ ↓ ↓ ↓
0 1 0 1	1 0 1 0 1

Remarque

- ✓ L'entier positif est représenté sous sa forme naturelle.
- ✓ L'entier négatif est représenté par le complément à 1.
- ✓ Dans cette représentation, le bit du poids fort nous indique le signe (0 : positif, 1 : négatif).
- ✓ Le complément à un du complément à un d'un nombre est égal au nombre lui-même $CA_1(CA_1(N)) = N$.
- ✓ On appelle complément à un d'un nombre N un autre nombre N' tel que : $N + N' = 2^n - 1$ tel que n est le nombre de bits de la représentation du nombre N. **Exemple** : Soit $N = 1010$ sur 4 bits donc son complément à un est : $N' = (2^4 - 1) - N$, $N' = (16 - 1)_{10} - (1010)_2 = (15)_{10} - (1010)_2 = (1111)_2 - (1010)_2 = (0101)_2$

Décimal	Binaire							
12	0	0	0	0	1	1	0	0
-12	1	1	1	1	0	0	1	1

Exemple

Quelle est la valeur décimale représentée par la valeur 101010 en CA_1 sur 6 bits ?

- Le bit poids fort indique qu'il s'agit d'un nombre négatif.
- Valeur = $-CA_1(101010) = -(010101)_2 = -(21)_{10}$

Si on travaille sur 3 bits :

- Dans cette représentation, le bit du poids fort nous indique le signe
- On remarque que dans cette représentation le zéro possède aussi une double représentation (+0 et -0).
- Sur 3 bits on remarque que les valeurs sont comprises entre -3 et +3

- ✓ $-3 \leq N \leq +3$
- ✓ $-(4-1) \leq N \leq +(4-1)$
- ✓ $-(2^2-1) \leq N \leq +(2^2-1)$
- ✓ $-(2^{(3-1)}-1) \leq N \leq +(2^{(3-1)}-1)$

Alors si on travaille sur n bits, l'intervalle des valeurs qu'on peut représenter en CA_1 :

$$-(2^{(n-1)}-1) \leq N \leq +(2^{(n-1)}-1)$$

Valeur en CA_1	Valeur en binaire	Valeur décimale
000	000	+0
001	001	+1
010	010	+2
011	011	+3
100	-011	-3
101	-010	-2
110	-001	-1
111	-000	-0



REPRESENTATION EN COMPLEMENT A DEUX (COMPLEMENT VRAI)

Le complément à 2 est le mode de représentation le plus utilisé en arithmétique binaire et donc dans les ordinateurs pour coder les nombres entiers. Dans cette représentation, les nombres positifs se représentent par leur valeur binaire naturelle. Par exemple +6 est représenté par 0000 0110 sur un format de 8 bits. Par contre la représentation des nombres négatifs s'obtient comme suit :

- ✓ On part de la représentation binaire naturelle de l'opposé arithmétique du nombre à coder (nombre positif),
- ✓ On calcule son complément à 1 (CA_1) ou complément restreint. Celui-ci est obtenu en inversant tous ses bits,
- ✓ On en déduit son complément à 2 (CA_2) ou complément vrai en ajoutant 1 au niveau du LSB (symbole de poids le plus faible (LSB : Least Significant Bit)).

Si on suppose que a est un nombre sur n bits alors : $a + 2^n = a \text{ modulo } 2^n$ et si on prend le résultat sur n bits on va obtenir la même valeur que a : $\rightarrow a + 2^n = a$

Exemple : soit $a = 1001$ sur 4 bits $2^4 = 10000$

$$\begin{array}{r} \\ + \\ \hline 1 \end{array}$$

Si on prend le résultat sur 4 bits on trouve la même valeur de $a = 1001$

- ✓ Si on prend deux nombres entiers a et b sur n bits, on remarque que la soustraction peut être ramené à une addition : $a - b = a + (-b)$, pour cela il suffit de trouver une valeur équivalente $a - b$?
- ✓ $a - b = a + 2^n - b = a + (2^n - 1) - b + 1$, et on a : $b + CA_1(b) = 2^n - 1$
Donc $CA_1(b) = (2^n - 1) - b$
- ✓ Si on remplace dans la première équation on obtient : $a - b = a + CA_1(b) + 1$
- ✓ La valeur $CA_1(b) + 1$ s'appelle le **complément à deux** de b : $CA_1(b) + 1 = CA_2(b)$
- ✓ Et enfin on va obtenir : $a - b = a + CA_2(b) \rightarrow$ transformer la soustraction en une addition.

Exemple

Trouver le complément vrai de : 11000101 sur 8 bits ?

$$CA_2(11000101) = CA_1(11000101) + 1$$

$$CA_1(11000101) = (00111010)$$

$$CA_2(11000101) = (00111010) + 1 = (00111011)$$

Remarques

- ✓ Pour trouver le complément à 2 d'un nombre : il faut parcourir les bits de ce nombre à partir du poids faible et garder tous les bits avant le premier 1 et inverser les autres bits qui viennent après.

$$\begin{array}{cccccccc} 1 & 1 & 0 & 0 & 0 & 1 & 0 & 1 \\ \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow \\ 0 & 0 & 1 & 1 & 1 & 0 & 1 & 1 \end{array} \quad \begin{array}{cccccccc} 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 \\ \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow \\ 1 & 0 & 1 & 0 & 1 & 1 & 0 & 0 \end{array}$$

- ✓ Dans cette représentation, le bit du poids fort nous indique le signe (0 : positif, 1 : négatif).
- ✓ Le complément à deux du complément à deux d'un nombre est égal au nombre lui-même :

$$CA_2(CA_2(N)) = N$$

Exemple :

Quelle est la valeur décimale représentée par la valeur 101010 en complément à deux sur 6 bits ?



- Le bit du poids fort indique qu'il s'agit d'un nombre négatif.
- Valeur = $-CA_2(101010) = -(010101 + 1)_2 = -(010110)_2 = -(22)_{10}$

Si on travaille sur 3 bits :

- Dans cette représentation, le bit du poids fort nous indique le signe.
- On remarque que le zéro n'a pas une double représentation

Sur 3 bits on remarque que les valeurs sont comprises entre -4 et +3

$$-4 \leq N \leq +3$$

$$-4 \leq N \leq +(4-1)$$

$$-2^2 \leq N \leq +(2^2-1)$$

$$-2^{(3-1)} \leq N \leq (2^{(3-1)}-1)$$

Si on travaille sur n bits, l'intervalle des valeurs qu'on peut représenter en CA_2 : $-(2^{(n-1)}) \leq N \leq +(2^{(n-1)}-1)$

Valeur en CA_2	Valeur en binaire	valeur
000	000	+0
001	001	+1
010	010	+2
011	011	+3
100	-100	-4
101	-011	-3
110	-010	-2
111	-001	-1

La représentation en complément à deux (complément à vrai) est la représentation la plus utilisée pour la représentation des nombres négatifs dans la machine.

En Général (intervalles)

Nb. de bits	Binaire					
	Non signés		Valeur signée		Complément à 2	
	Min	Max	Min	Max	Min	Max
n	0	$2^n - 1$	$-(2^{n-1} - 1)$	$2^{n-1} - 1$	-2^{n-1}	$2^{n-1} - 1$

LA REPRESENTATION DES NOMBRES REELS

Un nombre réel est constitué de deux parties : la partie entière et la partie fractionnelle (les deux parties sont séparées par une virgule)

- Problème : comment indiquer à la machine la position de la virgule ?
- Il existe deux méthodes pour représenter les nombres réels :
 - ✓ Virgule fixe : la position de la virgule est fixe
 - ✓ Virgule flottante : la position de la virgule change (dynamique).

LA VIRGULE FIXE

Un nombre en virgule fixe est une valeur munie ou non d'un signe enregistré comme un entier binaire (DCB). Un tel nombre est dit en virgule fixe car le soin de placer la virgule, revient au programmeur. C'est à dire que la virgule n'apparaît pas dans le stockage du nombre, mais sera placé par le programmeur qui utilise ce nombre, décomposant ainsi la valeur lue en une partie entière et une partie fractionnaire.

Dans cette représentation la partie entière est représentée sur n bits et la partie fractionnelle sur p bits, en plus un bit est utilisé pour le signe.

Exemple : si n=3 et p=2 on va avoir les valeurs suivantes :

Signe	P.E	P.F	valeur
0	000	00	+0,0
0	000	01	+0,25
0	000	10	+0,5
0	000	11	+0,75
0	001	.00	+1,0
.	.	.	.
.	.	.	.

Dans cette représentation les valeurs sont limitées et nous n'avons pas une grande précision



REPRÉSENTATION EN VIRGULE FLOTTANTE

A un nombre en virgule flottante, on associe deux jeux de valeurs. Le premier représente les chiffres significatifs du nombre, c'est la **mantisse**. Le second indique la puissance à laquelle la base est élevée, c'est l'**exposant**. Ainsi, la manière la plus évidente et la plus concise pour représenter un nombre en virgule flottante est donc d'employer un exposant et une mantisse signée. Chaque nombre réel peut s'écrire de la façon suivante : $N = \pm M * b^e$

- ✓ M : mantisse,
- ✓ b : la base,
- ✓ e : l'exposant

Exemple :

- ✓ $15,6 = 0,156 * 10^{+2} = 0,0156 * 10^{+3} = 0,00156 * 10^{+4}$
- ✓ $-(110,101)_2 = -(0,110101)_2 * 2^{+3} = -(0,0110101)_2 * 2^{+4}, \dots$
- ✓ $(0,00101)_2 = (0,101)_2 * 2^{-2}$

On peut noter qu'à priori, une représentation en virgule flottante n'est pas nécessairement unique. Il nous faut donc les représenter sous une forme normalisée afin que la représentation ne varie pas d'un matériel à l'autre. Une mantisse normalisée en virgule flottante, est un nombre dans lequel le premier chiffre après la virgule est différent de 0, alors que le premier chiffre avant la virgule est égal à 0. On peut trouver en fonction des organismes de normalisation ou des constructeurs plusieurs normes de représentation de nombres en virgule flottante.

- Cette représentation sur n bits :
 - La mantisse est sous la forme signe/valeur absolue
 - 1 bit pour le signe
 - k bits pour la valeur.
 - L'exposant (positif ou négatif) est représenté sur p bits.

Signe mantisse	Exposant	Mantisse normalisée
1 bit	p bits	k bits

- Pour la représentation de l'exposant on utilise :
 - Le complément à deux
 - Exposant décalé ou biaisé

REPRÉSENTATION DE L'EXPOSANT EN COMPLEMENT A DEUX

On veut représenter les nombres $(0,015)_8$ et $-(15,01)_8$ en virgule flottante sur une machine ayant le format suivant :

Signe mantisse	Exposant en CA2	Mantisse normalisée
1 bit	4 bits	8 bits

$$(0,015)_8 = (0,000001101)_2 = 0,1101 * 2^{-5}$$

- ✓ Signe mantisse : positif (0)
- ✓ Mantisse normalisée : 0,1101
- ✓ Exposant = -5 → utiliser le complément à deux pour représenter le -5 Sur 4 bits est $CA_2(0101) = 1011$

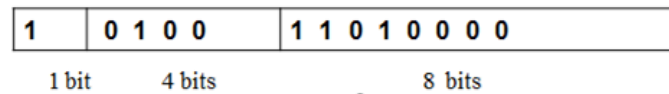
0	1 0 1 1	1 1 0 1 0 0 0 0
1 bit	4 bits	8 bits

$$-(15,01)_8 = -(001101,000001)_2 = -0,1101000001 * 2^4$$

- ✓ Signe mantisse : négatif (1)
- ✓ Mantisse normalisée : 0,1101000001
- ✓ Exposant = 4, en complément à deux il garde la même valeur (0100)



On remarque que la mantisse est sur 10 bits (1101 0000 01), et sur la machine seulement 8 bits sont utilisés pour la mantisse. Dans ce cas on va prendre les 8 premiers bits de la mantisse → la mantisse est tronquée : on va prendre uniquement 8 bits → perdre dans la précision



REPRESENTATION PAR L'EXPOSANT DECALÉ OU BIAISÉ

La technique de l'exposant décalé est très utile car elle évite de traiter des exposants négatifs (à condition bien sûr que le décalage choisi soit suffisamment grand).

En complément à 2, l'intervalle des valeurs qu'on peut représenter sur p bits est :

$-2^{(p-1)} \leq N \leq 2^{(p-1)} - 1$ si on rajoute la valeur $2^{(p-1)}$ à tous les termes de cette inégalité :

$-2^{(p-1)} + 2^{(p-1)} \leq N + 2^{(p-1)} \leq 2^{(p-1)} - 1 + 2^{(p-1)} \Rightarrow 0 \leq N + 2^{(p-1)} \leq 2^p - 1$

On pose $N' = N + 2^{(p-1)}$ donc : $0 \leq N' \leq 2^p - 1$

- Dans ce cas on obtient des valeurs positives.
- La valeur $2^{(p-1)}$ s'appelle le **biais** ou le **décalage**
- Ainsi, avec l'exposant biaisé on a transformé les **exposants négatifs** à des **exposants positifs** en rajoutons à l'exposant la valeur $2^{(p-1)}$.

Exposant Biaisé = Exposant réel + Biais

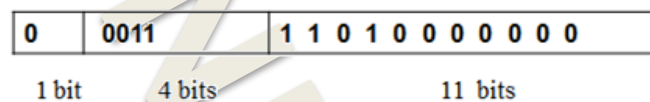
Exemple

On veut représenter les nombres $(0,015)_8$ et $-(15,01)_8$ en virgule flottante sur une machine ayant le format suivant :

Signe mantisse	Exposant décalé	Mantisse normalisée
1 bit	4 bits	11 bits

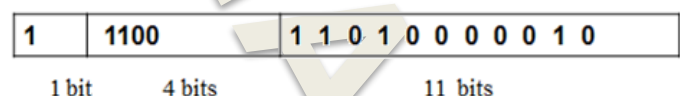
$(0,015)_8 = (0,000001101)_2 = 0,1101 \cdot 2^{-5}$

- ✓ Signe mantisse : positif (0)
- ✓ Mantisse normalisée : 0,1101
- ✓ Exposant réel = -5
- ✓ Calculer le biais : $b = 2^{(4-1)} = 8$
- ✓ Exposant Biaisé = $-5 + 8 = +3 = (0011)_2$



$-(15,01)_8 = (001101,000001)_2 = 0,1101000001 \cdot 2^4$

- ✓ Signe mantisse : négatif (1)
- ✓ Mantisse normalisée : 0,1101000001
- ✓ Exposant réel = +4
- ✓ Calculer le biais : $b = 2^{(4-1)} = 8$
- ✓ Exposant Biaisé = $4 + 8 = +12 = (1100)_2$



REPRESENTATION PAR LA NORME IEEE 754

La norme IEEE 754 (que l'on peut prononcer « i 3 e » : Institute of Electrical and Electronics Engineers) définit la façon de coder un nombre réel. Cette norme se propose de coder le nombre sur 32 bits et définit trois composantes :

- ✓ Le signe est représenté par **un seul bit**, le bit de poids fort.
- ✓ L'exposant est codé sur les **8 bits** consécutifs au signe.
- ✓ La mantisse (les bits situés après la virgule) sur les **23 bits** restants.



Certaines conditions sont toutefois à respecter pour les exposants :

- ✓ L'exposant 00000000 est interdit.
- ✓ L'exposant 11111111 est interdit. On s'en sert toutefois pour signaler des erreurs, on appelle alors cette configuration du nombre NaN, ce qui signifie « Not a number ».
- ✓ La mantisse sera sous la forme 1, partie fractionnaire → qu'on appelle pseudo-mantisse. → La pseudo-mantisse de la norme IEEE 754 est donc constituée implicitement d'un 1 et de la virgule, suivis des 23 ou 52 bits effectifs virgule.
- ✓ Il faut rajouter $2^8-1=127$ (01111111) à l'exposant (Exposant Biaisé) pour une conversion de décimal vers un nombre réel binaire. Les exposants peuvent ainsi aller de -254 à 255.

Exemple

- ✓ Traduisons en binaire, en utilisant la norme IEEE 754, le nombre -6,625.
- ✓ Codons d'abord la valeur absolue en binaire : $(6,625)_{10} = (110,1010)_2$
- ✓ Nous mettons ce nombre sous la forme : 1, partie fractionnaire (pseudo-mantisse)
- ✓ Pseudo-mantisse : $110,1010 = 1,101010 \cdot 2^2$ (2^2 décale la virgule de 2 chiffres vers la droite)
- ✓ La partie fractionnaire étendue sur 23 bits est donc 101 0100 0000 0000 0000 0000.
- ✓ Exposant = $127 + 2 = (129)_{10} = (1000\ 0001)_2$



En hexadécimal : 00D4 00 00.

	Simple précision	Double précision
Taille totale	32	64
Bit de signe	1	1
Bits d'exposant	8	11
Bits de significande	23	52
Domaine décimal	$\approx 10^{-38}$ à 10^{+38}	$\approx 10^{-308}$ à 10^{+308}

OPERATIONS ARITHMETIQUES EN CA₂

Exemple1 : Supposons qu'on travaille sur 5 bits ($-16 \leq N \leq 15$)

Positive Addition:

+ 9	0 1 0 0 1
+ 4	0 0 1 0 0
+ 13	0 1 1 0 1

Le résultat est positif
 $(01101)_2 = (+13)_{10}$

Positive Addition with Carry:

+ 9	0 1 0 0 1
- 4	1 1 1 0 0
+ 5	1 0 0 1 0 1

Report → (Carry out of the 5th bit)
Le résultat est positif
 $(00101)_2 = (+5)_{10}$

Negative Addition:

- 9	1 0 1 1 1
- 4	1 1 1 0 0
- 13	1 1 0 0 1 1

Report → (Carry out of the 5th bit)
Le résultat est négatif
 $-CA_2(10011)_2 = -(01101)_2 = -(13)_{10}$

Zero Result:

+ 9	0 1 0 0 1
- 9	1 0 1 1 1
+ 0	1 0 0 0 0 0

Report → (Carry out of the 5th bit)
Le résultat est positif
 $(00000)_2 = (0)_{10}$



LA RETENUE ET LE DEBORDEMENT

Une limitation intrinsèque des systèmes logiques est, comme nous avons pu le mentionner, la longueur des mots dans un code. Dans le cas des codes analytiques, cette limitation n'est pas à écarter et doit être considérée avec prudence. Un problème important survient dans le cas de l'**addition** et la **soustraction** lorsque le résultat ne peut être représenté selon la longueur des mots binaires.

Supposons que nous disposions d'un code de 4 bits pour représenter les nombres en binaire signé. Cela signifie que nous disposons des nombres allant de -8 à +7. Si le résultat de l'opération dépasse ces limites, nous faisons face à un cas de **débordement**. Par exemple, si nous additionnons +4 et +4, le résultat est +8 et n'est pas représentable en complément à 2 sur 4 bits. Il en est de même lorsque nous additionnons -4 et -5.

NOTION DE CARRY = RETENUE

Lors d'une opération arithmétique effectuée sur des nombres de p bits, un $(p+1)^{\text{ème}}$ bit peut être généré (bit de carry). Ainsi, On dit qu'il y a une **retenue** si une opération arithmétique génère un **report**.

NOTION D'OVERFLOW OU DE DEPASSEMENT DE CAPACITE

Les **dépassements** surviennent uniquement lors d'une opération arithmétique mettant en jeu des nombres de **p bits et de même signe**, le résultat peut se révéler être **trop grand** ou **trop petit** pour être représentable par la machine. On peut les reconnaître par l'**incohérence du résultat final** qui se trouve être négatif dans le cas de l'addition de deux nombres positifs, et négatif dans le cas de l'addition de deux nombres positifs. Ainsi :

- On dit qu'il y a un **débordement** (Overflow) ou dépassement de capacité : si le résultat de l'opération sur **n bits** est **faux (erroné)** :
 - ✓ Le nombre de bits utilisés est insuffisant pour contenir le résultat → Autrement dit le résultat dépasse l'intervalle des valeurs sur les **n bits utilisés**.
 - ✓ Nous avons un débordement si la somme de deux nombres positifs donne un nombre négatif. Ou la somme de deux nombres négatifs donne un Nombre positif.
 - ✓ Il n'y a jamais un débordement si les deux nombres sont de signes différents.

Exemple 2 : Supposons qu'on travaille sur 4bits ($-8 \leq N \leq 7$)

$$\begin{array}{r} +4 \\ +2 \\ \hline +6 \end{array} \quad \begin{array}{r} + \quad 0100 \\ + \quad 0010 \\ \hline 0110 \end{array} \quad (+6)$$

Pas de retenue
Pas de dépassement

$$\begin{array}{r} +4 \\ +6 \\ \hline +10 \end{array} \quad \begin{array}{r} + \quad 0100 \\ + \quad 0110 \\ \hline 1010 \end{array} \quad (-6)$$

Pas de retenue
Il y a dépassement
Résultat erroné

$$\begin{array}{r} -4 \\ -2 \\ \hline -6 \end{array} \quad \begin{array}{r} + \quad 1100 \\ + \quad 1110 \\ \hline 11010 \end{array} \quad (-6)$$

Il y a retenue
Pas de dépassement
Ignorer la retenue
Résultat correct

$$\begin{array}{r} -4 \\ -6 \\ \hline -10 \end{array} \quad \begin{array}{r} + \quad 1100 \\ + \quad 1010 \\ \hline 10110 \end{array} \quad (+6)$$

Il y a retenue
Il y a dépassement
Ignorer la retenue
Résultat erroné

